Python Lesson: Arrays and How They Differ From Lists

Introduction

When writing programs, you often need to store **multiple values together**. For example, you might want to store:

- Several numbers
- Multiple student names
- A collection of prices

Instead of creating many separate variables, Python allows you to store many values inside a **single data structure**.

Two structures that can store multiple values are:

- **Lists**
- **Arrays**

Although beginners often use **lists instead of arrays**, these two structures are **not the same**. Understanding their differences helps you choose the correct structure when writing programs.

What Is a List?

A **list** is a collection that stores multiple values inside a single variable. Lists are built directly into Python and are one of the most commonly used data structures.

All elements can be the different data types.

Creating a List

Lists use **square brackets**.

Example:

Python

```python
numbers = [10, 20, 30, 40]
```

This list contains four numbers.

Lists can also store **different types of data**.

Example:

Python

```python
data = ["Alex", 25, 4.5, True]
```

This list contains:

Value	Data Type
"Alex"	String
25	Integer
4.5	Float
True	Boolean

This flexibility is one reason lists are widely used.

What Is an Array?

An **array** is another structure that stores multiple values. However, arrays have an important restriction:

All elements must be the same data type.

Arrays are provided through Python's built-in **array module**.

Example:

```python
from array import array

numbers = array('i', [10, 20, 30, 40])
```

Explanation:

- `'i'` tells Python the array stores **integers**
- The list `[10, 20, 30, 40]` provides the values

If you try to insert a different type, Python will produce an error.

Example that would fail:

```python
array('i', [10, "apple"])
```

This fails because `"apple"` is a string.

Accessing Array Elements

Arrays also use **index positions**.

Example:

```python
from array import array

numbers = array('i', [10, 20, 30])

print(numbers[1])
```

Output

```
20
```

The second element is printed because indexing starts at **0**.

Modifying an Array

You can change values in an array.

Example:

```python
from array import array

numbers = array('i', [5, 10, 15])

numbers[1] = 50

print(numbers)
```

Output

```
array('i', [5, 50, 15])
```

Why Arrays Exist

Arrays are designed to store **large collections of the same type of data efficiently**.

This makes them useful for:

- Numeric calculations
- Scientific computing
- Large datasets

Libraries such as NumPy provide advanced arrays that are widely used in data science and machine learning.

Major Differences Between Lists and Arrays

Feature	List	Array
Data Types	Can store mixed types	Must store one type
Built-in	Yes	Requires array module
Flexibility	Very flexible	More restrictive
Memory Usage	Uses more memory	More memory efficient

Common Use	General programming	Numeric data processing

Example Comparison

List Example

Python Run

```python
items = [10, "banana", 3.5]

print(items)
```

Output

```
[10, 'banana', 3.5]
```

The list stores **different types of values**.

Array Example

Python Run

```python
from array import array

numbers = array('i', [1, 2, 3])

print(numbers)
```

Output

```
array('i', [1, 2, 3])
```

The array stores **only integers**.

When to Use Lists vs Arrays

Use Lists When

- You need flexibility
- Data types may vary
- You want Python's most common structure

Lists are used in **most Python programs**.

Use Arrays When

- All data is the same type
- Memory efficiency matters
- You are working with large numeric datasets

Arrays are commonly used in **mathematics and scientific programming**.

Summary Table

Concept	Explanation	Example
List	Flexible collection that stores multiple types	[1, "apple", 3.5]
Array	Collection storing only one type	array('i', [1,2,3])
Index	Position of an element	list[0]
Modify Value	Change item at a position	numbers[1] = 50
Append	Add value to list	list.append(4)